IST 387: Introduction to Applied Data Science
Lab 10: Notes

Notes Prepared by: Dr. Preeti Jagadev

Reference: Data Science for Business with R, written by Jeffrey S. Saltz and Jeffrey M. Stanton

Contents of the Notes:

- Introduction
- Association Rules Data
- Association Rules Mining
- Exploring how the Association Rules Algorithm works
- Building Association Rules in R
- Important R Functions

Introduction

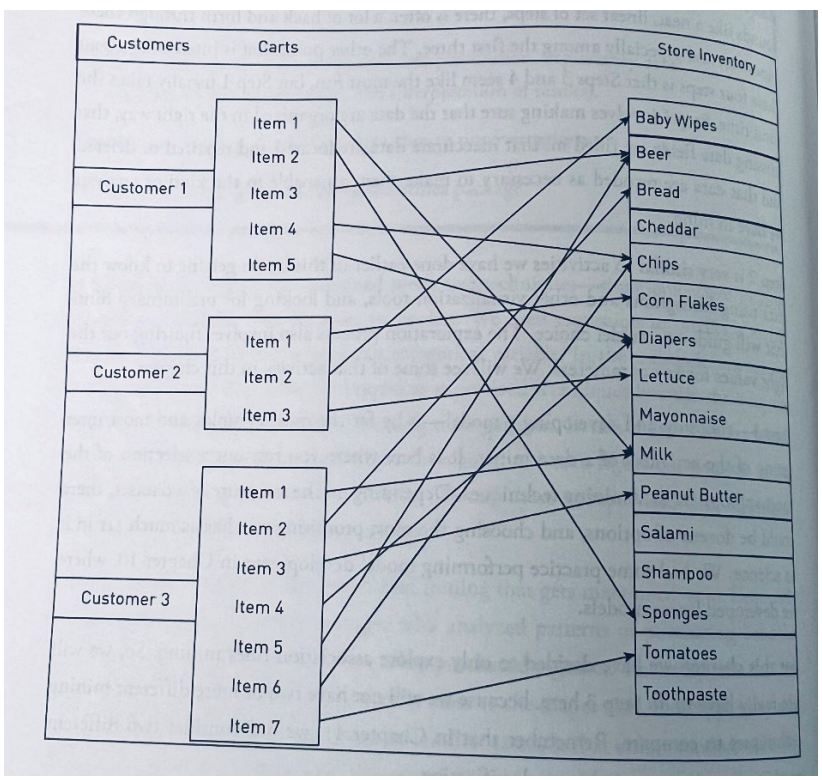
Unsupervised Learning uses machine learning algorithms to analyze unlabeled data sets. These algorithms discover hidden patterns in data without the need for human intervention (hence, they are "unsupervised").

Data mining typically consists of four processes:

1. **Data preparation:** Makes sure that the data is organized in the right way, missing data fields are filled in, inaccurate data is located and repaired or deleted.
2. **Exploratory data analysis:** Getting to know the data using histograms and other visualization tools, and looking for preliminary hints that will guide our model choice. The exploration process also involves figuring out the right values for key parameters.
3. **Model development**: By far the most complex and most interesting of the activities of a data miner. It is here where you test out a selection of the most appropriate data mining techniques. Depending on the structure of a dataset, there could be dozens of options, and choosing the most promising one has as much art in it as science.
4. **Interpretation of results:** Focuses on making sense out of what the data mining algorithm has produced. This is the most important step from the perspective of the data user because this is where an actionable conclusion is formed.

Association Rules Data

Look at the following figure with all the boxes and arrows:



From the figure, you can see that each supermarket customer has a grocery cart that contains several items from the larger set of items that the grocery store stocks. The **association rules algorithm** tries out many if-then propositions, such as, "If diapers are purchased, then beer is also purchased." The algorithm uses a dataset of transactions to evaluate a long list of these rules for a value known as **support**.

Support is the proportion of times that a particular combination occurs across all shopping carts. The algorithm also evaluates another quantity called **confidence,** which is how frequently a particular if-then rule occurs among all the times when the "if" item is present.

In the output from R's **association rule algorithm**, the "if" side of the rule is referred to as the "left-hand side" (LHS) and the "then" side of the rule is referred to as the "right-hand side" (RHS). The RHS is always just one item, for example, diapers. The LHS, however, can have more than one item in it. In the diapers-beer example, our LHS was just diapers, but we might also find a rule that had additional items on the LHS.

Association Rules Mining

We can get started with association rules mining very easily using the R package known as **arules**:

```
install.packages("arules")
library(arules)
```

We will begin our exploration of association rules mining using a dataset that is built in to the **arules** package. We will use the Groceries dataset:

```
data(Groceries)
summary(Groceries)
```

The output is as follows:

```
transactions as itemMatrix in sparse format with
 9835 rows (elements/itemsets/transactions) and
 169 columns (items) and a density of 0.02609146

most frequent items:
      whole milk other vegetables       rolls/buns             soda           yogurt
            2513             1903             1809             1715             1372
         (Other)
           34055

element (itemset/transaction) length distribution:
sizes
   1    2    3    4    5    6    7    8    9   10   11   12   13   14   15   16   17   18
2159 1643 1299 1005  855  645  545  438  350  246  182  117   78   77   55   46   29   14
  19   20   21   22   23   24   26   27   28   29   32
  14    9   11    4    6    1    1    1    1    3    1

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  1.000   2.000   3.000   4.409   6.000  32.000

includes extended item information - examples:
       labels  level2           level1
1 frankfurter sausage meat and sausage
2     sausage sausage meat and sausage
3  liver loaf sausage meat and sausage
```

We see that Groceries is an **itemMatrix** object in sparse format. We have a rectangular data structure with 9,835 rows and 169 columns, where each row is a list of items that might appear in a grocery cart. The columns are the individual items that could be in the cart. A little later in the output, we see that there are 169 columns, which means that there are 169 items.

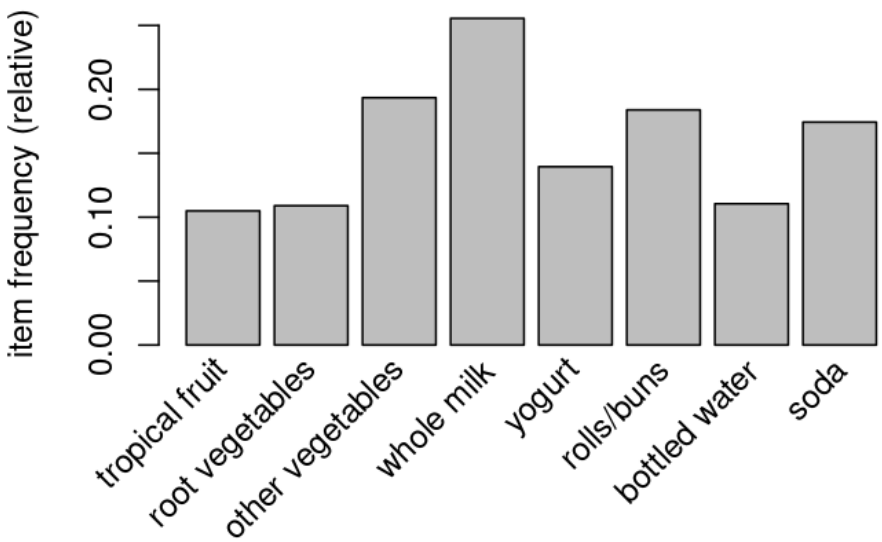
The reason the matrix is called sparse is that very few of these items exist in any given grocery basket (row). When an item appears in a basket, its cell contains a one (1), whereas if an item is not in a basket, is cell contains a zero (0). In any given row, most of the cells are zero and very few are one, and this is what is meant by sparse.

We can see from the Min, Median, Mean, and Max output that every cart has at least one item, half the carts have more than three items, the average number of items in a cart is 4.4, and the maximum number of items in a cart is 32. The output also shows us which items occur in grocery baskets most frequently.

Another way of inspecting our sparse matrix is with the item **Frequency Plot function**. This produces a bar graph that is similar in concept to a histogram: it shows the relative frequency of occurrence of different items in the matrix. When using the **itemFrequencyPlot( )** function, you must specify the minimum level of **support** needed to include an item in the plot. We can make a guess as to what level of **support** to choose based on the results of the **summary function**.

```
itemFrequencyPlot(Groceries, support=0.1)
```

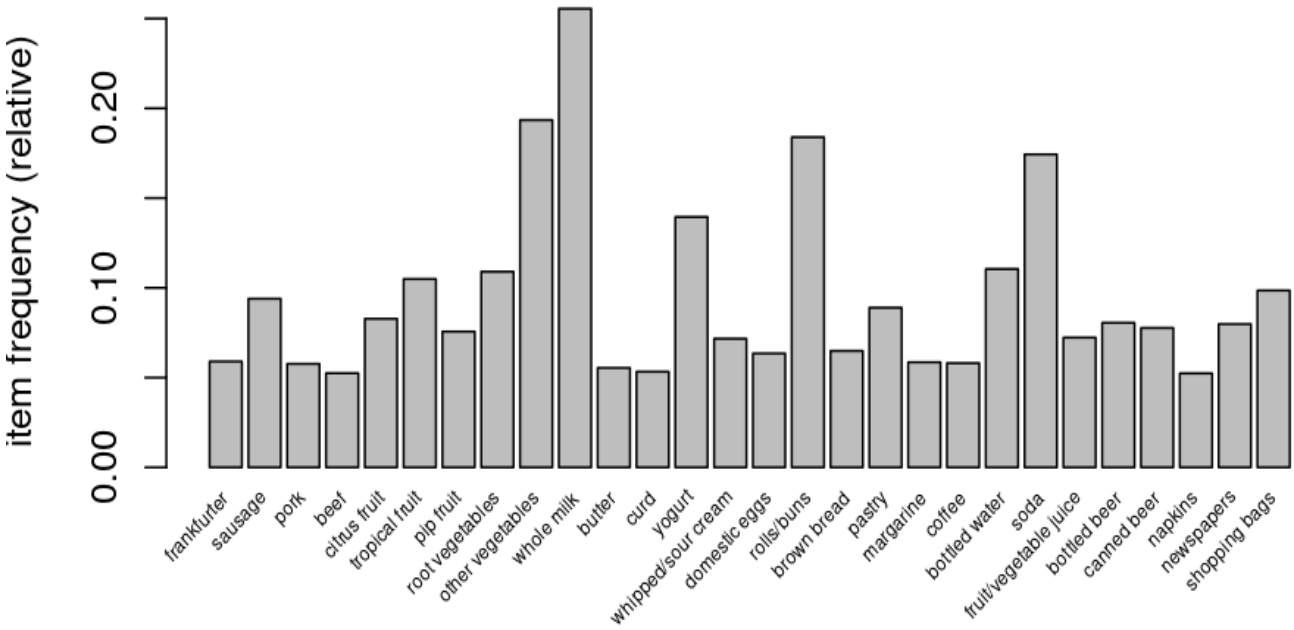
The output is as follows:



You should experiment with using different levels of support, just so that you can get a sense of the other common items in the dataset. If you show more than about 10 items, you will find that the labels on the X-axis start to overlap and obscure one another. Use the **cex.names** parameter to turn down the font size on the labels. This will keep the labels from overlapping at the expense of making the font size much smaller. Here's an example:

```
itemFrequencyPlot(Groceries, support=0.05, cex.names=0.5)
```

The output is as follows:



As you can see, this command yields about 25 items on the X-axis. Without worrying too much about the labels, you can also experiment with lower values of support, just to get a feel for how many items appear at the lower frequencies.

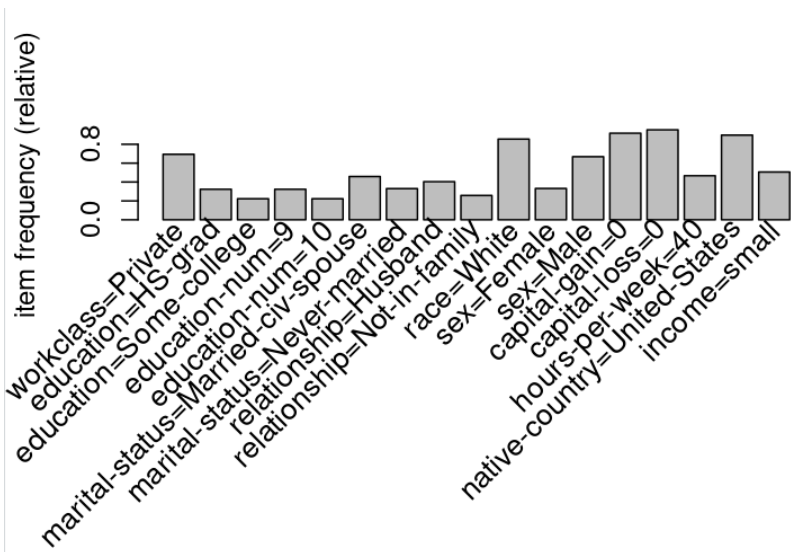
Now, we want to focus our attention on items that occur with some meaningful frequency in the dataset. Whether this is 1% or 0.005%, or something somewhat larger or smaller will depend on the size of the dataset and the intended use of the rules. Before we generate our first set of rules, let's take a step back and explore the type of data object that **itemFrequencyPlot** (and the other associated functions with arules) needs.

You might have noticed that it is not a dataframe (you can see this by doing the str command on the Groceries dataset). In fact, you can see it is a transactions dataset. But what if we have a dataframe: can we convert it to a transactions dataset? Yes, sometimes this is possible, but it depends on the dataframe.

Let's look at a different dataframe and convert it to a transactions dataset. To do this, we will use **the AdultUCI dataset**. In any event, you can see that we first need to convert any numbers or strings into factors. Also note that since some of the column names have a dash (-), we need to quote that column name, so R does not get confused and think, for example, that we want to subtract num from **AdultUCI$education**. After doing all this work, we can finally convert the dataframe into a transactions dataset, which we can then use to generate an item **FrequencyPlot.**

```
data(AdultUCI)
AdultUCI.t <- AdultUCI
AdultUCI.t$age <- as.factor(AdultUCI.t$age)
AdultUCI.t$fnlwgt <- as.factor(AdultUCI.t$fnlwgt)
AdultUCI.t$'education-num' <- as.factor(AdultUCI.t$'education-num')
AdultUCI.t$'capital-gain'  <- as.factor(AdultUCI.t$'capital-gain')
AdultUCI.t$'capital-loss'  <- as.factor(AdultUCI.t$'capital-loss')
AdultUCI.t$'hours-per-week'  <- as.factor(AdultUCI.t$'hours-per-week')
AdultUCI.trans  <- as(AdultUCI.t, "transactions")
itemFrequencyPlot(AdultUCI.trans, support=0.2, cex.names=1.1)
```

The output is as follows:



We have copied **AdultUCI** into **AdultUCl.t** so that we don't disturb the original dataset. Then we do several transformations to make some of the variables, such as age, into factor variables. Every variable that comes from a dataframe and ends up in a transactions database must somehow become a binary attribute. The **as()** function in the second to last line helps to make those transformations. After converting the dataframe to a transactions dataset, we display the item frequency plot.

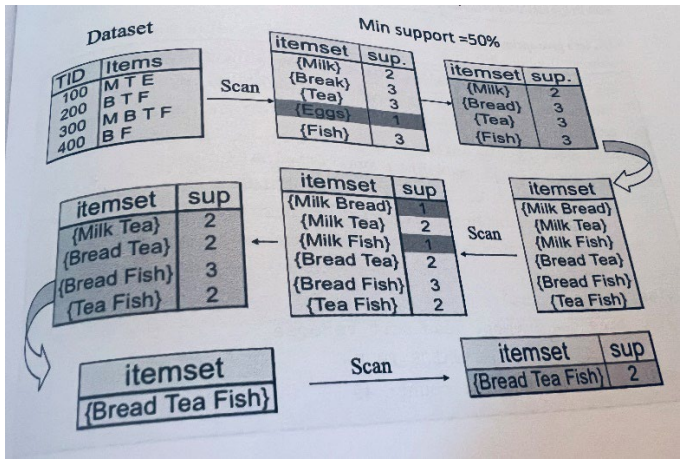
Now that we understand a little more about the transactions dataset, we are ready to generate some rules with the **apriori()** command. The term "apriori" refers to the specific algorithm that R will use to scan the dataset for appropriate rules. **Apriori** is a very commonly used algorithm and it is quite efficient at finding rules in transaction data.

Exploring How the Association Rules Algorithm Works

If you are wondering how **apriori** works, well, as shown in the following figure, apriori uses an iterative approach known as **level-wise search**. Basically, at the **first level**, we have individual items such as bread or milk (these are known as one item sets) that are frequently found (i.e., have sufficient support, as defined in the apriori command).

At the next level, the two-item sets we need to consider must have the property that each of their subsets must also have sufficient support. The algorithm keeps going up levels until there are no more combinations to be found. So, for example, as shown in our following example diagram, if

we know at Level 2 that the sets {Milk Tea}, {Bread Tea}, {Bread Fish}, and {Tea Fish} are the only sets with sufficient support, then at Level 3 we join these with each other to produce {Milk Tea Bread}, {Milk Tea Fish}, {Milk Bread Fish}, and {Bread Tea Fish}. But we only need to consider {Bread Tea Fish}, since the others each have subsets with insufficient support-such as {Milk Fish} or {Milk Bread}.



Apriori rules are in the form of "**If LHS, then RHS**." The abbreviation LHS means “left-hand side” and, RHS means “right-hand side.” So, each rule states that when the things on the LHS of the equation occur, the thing on the RHS occurs a certain percentage of the time.

Support for a rule refers to the frequency of co-occurrence of both members of the pair, that is, LHS and RHS together. The **confidence** of a rule refers to the proportion of the time that LHS and RHS occur together versus the total number of appearances of LHS. For example, if Milk and Bread occur together in 10% of the grocery cart (i.e., support), and Milk (by itself, ignoring Bread) occurs in 25% of the carts, then the confidence of the Milk/Bread rule is 0.10/0.25 = 0.40.

Building Association Rules in R

Let’s give apriori a try:

```
apriori(Groceries, parameter=list(support=0.005, confidence=0.5))
```

The output is as follows:

```
Apriori

Parameter specification:
 confidence minval smax arem  aval originalSupport maxtime support minlen
        0.5    0.1    1 none FALSE            TRUE       5   0.005      1
 maxlen target  ext
     10  rules TRUE

Algorithmic control:
 filter tree heap memopt load sort verbose
    0.1 TRUE TRUE  FALSE TRUE    2    TRUE

Absolute minimum support count: 49

set item appearances ...[0 item(s)] done [0.00s].
set transactions ...[169 item(s), 9835 transaction(s)] done [0.00s].
sorting and recoding items ... [120 item(s)] done [0.00s].
creating transaction tree ... done [0.00s].
checking subsets of size 1 2 3 4 done [0.00s].
writing ... [120 rule(s)] done [0.00s].
creating S4 object  ... done [0.00s].
set of 120 rules
```

We set up the **apriori command** to use a **support** of 0.005 and **confidence** of 0.5. The **minlen** and **maxlen** parameters refer to the minimum and maximum length of an item set that will be considered in generating rules. Obviously, you can't generate a rule unless you have at least one item in an item set and setting **maxlen** to 10 ensures that we will not have any rules that contain more than 10 items. If you recall from earlier in the chapter, the average cart only has 4.4 items, so we are not very likely to produce rules involving more than 10 items. In fact, a little later in the apriori output above, we see that the apriori algorithm only had to examine subsets of size one, two, three, and four. Apparently, no rule in this output contains more than four items. At the very end of the output, we see that 120 rules were generated. Let's store the resulting rules in a data structure called ruleset:

```
ruleset <- apriori(Groceries, parameter = list(support = 0.01,
                                               confidence=0.5))
```

If you examine the output from this command, you should find that we have slimmed down to 15 rules, quite a manageable number to examine one by one. We can get a preliminary look at the rules using the summary function, like this:

```
summary(ruleset)
```

The output is as follows:

```
set of 15 rules

rule length distribution (lhs + rhs):sizes
 3
15

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
      3       3       3       3       3       3

summary of quality measures:
    support          confidence        coverage            lift
 Min.   :0.01007   Min.   :0.5000   Min.   :0.01729   Min.   :1.984
 1st Qu.:0.01174   1st Qu.:0.5151   1st Qu.:0.02089   1st Qu.:2.036
 Median :0.01230   Median :0.5245   Median :0.02430   Median :2.203
 Mean   :0.01316   Mean   :0.5411   Mean   :0.02454   Mean   :2.299
 3rd Qu.:0.01403   3rd Qu.:0.5718   3rd Qu.:0.02598   3rd Qu.:2.432
 Max.   :0.02227   Max.   :0.5862   Max.   :0.04342   Max.   :3.030
     count
 Min.   : 99.0
 1st Qu.:115.5
 Median :121.0
 Mean   :129.4
 3rd Qu.:138.0
 Max.   :219.0

mining info:
      data ntransactions support confidence
 Groceries          9835    0.01        0.5
                                                                          call
 apriori(data = Groceries, parameter = list(support = 0.01, confidence = 0.5))
```

Looking under summary of quality measures, we have an overview of the distributions of **support, confidence,** and a value called **lift. Lift** considers the **support** for a rule, but also gives more weight to rules where the LHS and/or the RHS occurs less frequently. In other words, lift favors situations where LHS and RHS are not abundant but where the relatively few occurrences always happen together. The larger the value of **lift**, the more interesting the rule might be. Now we are ready to take a closer look at the rules we generated. The **inspect()** command gives us the detailed contents of the data object generated by **apriori( ):**

```
inspect(ruleset)
```

The output is as follows:

```
     lhs                                      rhs                support
[1]  {curd, yogurt}                        => {whole milk}       0.01006609
[2]  {other vegetables, butter}            => {whole milk}       0.01148958
[3]  {other vegetables, domestic eggs}     => {whole milk}       0.01230300
[4]  {yogurt, whipped/sour cream}          => {whole milk}       0.01087951
[5]  {other vegetables, whipped/sour cream} => {whole milk}       0.01464159
[6]  {pip fruit, other vegetables}         => {whole milk}       0.01352313
[7]  {citrus fruit, root vegetables}       => {other vegetables} 0.01037112
[8]  {tropical fruit, root vegetables}     => {other vegetables} 0.01230300
[9]  {tropical fruit, root vegetables}     => {whole milk}       0.01199797
[10] {tropical fruit, yogurt}              => {whole milk}       0.01514997
[11] {root vegetables, yogurt}             => {other vegetables} 0.01291307
[12] {root vegetables, yogurt}             => {whole milk}       0.01453991
[13] {root vegetables, rolls/buns}         => {other vegetables} 0.01220132
[14] {root vegetables, rolls/buns}         => {whole milk}       0.01270971
[15] {other vegetables, yogurt}            => {whole milk}       0.02226741
     confidence coverage   lift     count
[1]  0.5823529  0.01728521 2.279125  99
[2]  0.5736041  0.02003050 2.244885 113
[3]  0.5525114  0.02226741 2.162336 121
[4]  0.5245098  0.02074225 2.052747 107
[5]  0.5070423  0.02887646 1.984385 144
[6]  0.5175097  0.02613116 2.025351 133
[7]  0.5862069  0.01769192 3.029608 102
[8]  0.5845411  0.02104728 3.020999 121
[9]  0.5700483  0.02104728 2.230969 118
[10] 0.5173611  0.02928317 2.024770 149
[11] 0.5000000  0.02582613 2.584078 127
[12] 0.5629921  0.02582613 2.203354 143
[13] 0.5020921  0.02430097 2.594890 120
[14] 0.5230126  0.02430097 2.046888 125
[15] 0.5128806  0.04341637 2.007235 219
```

You can see that each of the 15 rules shows the LHS, the RHS, the **support,** the **confidence**, and the **lift**. Rules 7 and 8 have the highest level of **lift:** the fruits and vegetables involved in these two rules have a relatively low frequency of occurrence, but their **support** and **confidence** are both relatively high.

Contrast these two rules with Rule 1, which also has high **confidence**. The reason for this contrast is that milk is a frequently occurring item, so there is not much novelty to that rule. On the other hand, the combination of fruits, root vegetables, and other vegetables suggests a need to find out more about customers whose carts contain only vegetarian or vegan items.

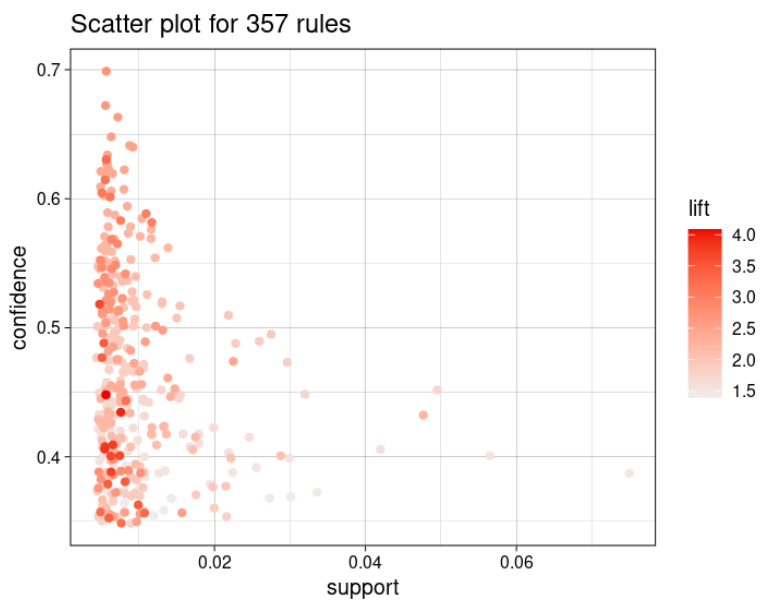
Now it is possible that we have set our parameters for **confidence** and **support** too stringently and as a result we have missed some truly novel combinations that might lead us to better insights. We can use a data visualization package to help explore this possibility. The R package called **arulesViz** has methods of visualizing the rule sets generated by **apriori( )** that can help us examine a larger set of rules. First, install and library **the arulesViz package**:

```
install.packages("arulesViz")
library(arulesViz)
ruleset <- apriori(Groceries, parameter = list(support=0.005,
                                               confidence=0.35))
```

When you run this command, you should find that you now generate 357 rules. That is way too many rules to examine manually, so let's use the **arulesViz** package to see what we have. We will use the plot command, so that when the **plot( )** command receives a data object of type **rules** (as generated by **apriori)**, it will use custom code that is built into **arulesViz** to do the plot:

```
plot(ruleset)
```

The output is as follows:



Even though we see a two-dimensional plot, we have three variables represented here. **Support** is on the X-axis and **confidence** is on the Y-axis. We would like to have rules that have **high support** and **high confidence**. We know, however, that **lift** serves as a measure of interestingness, and we are also interested in the rules with the **highest lift**. On this plot, the **lift** is shown by the darkness of a dot that appears on the plot. The darker the dot, the closer the lift of that rule is to 4.0, which appears to be the highest lift value among these 357 rules.

The other thing we can see from this plot is that although the **support** of rules ranges from somewhere below 1% all the way up above 7%, all the rules with high **lift** seem to have **support** below 1%. On the other hand, there are rules with high **lift** and high **confidence,** which sounds quite positive.

Based on this evidence, lets focus on a smaller set of rules that have only the very highest levels of lift. The following command makes a subset of the larger set of rules by choosing only those rules that have lift higher than 3.5:

```
goodrules <- ruleset[quality(ruleset)$lift > 3.5]
```

Note that the use of the square brackets with our data structure **ruleset** allows us to index only those elements of the data object that meet our criteria. In this case, we use the expression **quality (ruleset)$lift** to tap into the **lift** parameter for each rule. The inequality **test > 3.5** gives us just those rules with the highest **lift**. When you run this line of code, you should find that **goodrules** contains just nine rules. Let's inspect those nine rules:

```
    lhs                      rhs                  support confidence   coverage     lift count
[1] {herbs}             => {root vegetables} 0.007015760  0.4312500 0.01626843 3.956477    69
[2] {onions,
     other vegetables}  => {root vegetables} 0.005693950  0.4000000 0.01423488 3.669776    56
[3] {beef,
     other vegetables}  => {root vegetables} 0.007930859  0.4020619 0.01972547 3.688692    78
[4] {tropical fruit,
     curd}              => {yogurt}          0.005287239  0.5148515 0.01026945 3.690645    52
[5] {citrus fruit,
     pip fruit}         => {tropical fruit}  0.005592272  0.4044118 0.01382816 3.854060    55
[6] {pip fruit,
     other vegetables,
     whole milk}        => {root vegetables} 0.005490595  0.4060150 0.01352313 3.724961    54
[7] {citrus fruit,
     other vegetables,
     whole milk}        => {root vegetables} 0.005795628  0.4453125 0.01301474 4.085493    57
[8] {root vegetables,
     whole milk,
     yogurt}            => {tropical fruit}  0.005693950  0.3916084 0.01453991 3.732043    56
[9] {tropical fruit,
     other vegetables,
     whole milk}        => {root vegetables} 0.007015760  0.4107143 0.01708185 3.768074    69
```

When you look over these rules, it seems evident that shoppers are purchasing combinations of items that go together in recipes. The first three rules really seem like ingredients for soup! Rules 4 and 5 seem like a fruit platter with dip. The other four rules might also connect to a recipe, although it is not quite as obvious what.

If we were to present these results to a store manager, we might recommend that recipes could be published along with coupons and popular recipes, such as for homemade soup, and to have all of the ingredients grouped together in the store along with signs saying, "Mmmm, homemade soup!

Important R Functions

- **apriori( ):** Uses the algorithm of the same name to analyze a transaction dataset and generates rules.
- **as.factor( ):** Coerce a variable [usually a string] into a factor.
- **data( ):** Makes a data set from a package available.

- **itemFrequencyPlot( ):** Shows the relative frequency of commonly occurring items in the spare occurrence matrix.
- **inspect( ):** Shows the contents of the data object generated by apriori( ) that generates the association rules.
- **install.packages( ):** Loads package from the CRAN repository.
- **list( ):** Create an R list object froma set of data objects.
- **plot( ):** Generic plotting function that can be customized by packages like **arulesViz.**
- **summary( ):** Provides an overview of the contents of a data structure.